

Design of a 4-bit ALU Challenge Report

Team name: The A Factor

Team Members:

Abhinav Sundar Davala

Apoorv Mathur

Aneesh Vashisht

This report documents the design and implementation of an 8-operation 4-bit Arithmetic Logic Unit (ALU) developed for the Design Challenge Round 1. The ALU is implemented in Verilog HDL and verified using simulation waveforms. Each module performs a unique digital operation, combining arithmetic, logical, and data encoding capabilities.

1. Design Assumptions:

1. The ALU operates on **4-bit unsigned inputs (A, B)**.
2. Division by zero returns a **quotient and remainder of 0**.
3. Arithmetic overflow (beyond 4 bits) is handled by **carry/borrow flags**.
4. Logic operations are purely **bitwise**, not arithmetic.
5. Parity generation uses **even parity** convention.
6. Comparator outputs a **3-bit flag vector**:
 - $CMP[0] = A == B$
 - $CMP[1] = A > B$
 - $CMP[2] = A < B$
7. Simulation and waveforms assume **unit delay (#10)** for visual clarity.

2. Design Approach

Objective:

To design a modular and synthesizable **4-bit modular ALU** in Verilog HDL capable of performing **eight distinct operations** across arithmetic, logical, and data-encoding domains.

Approach Summary:

- Followed **modular and hierarchical** design — each function (add, sub, mul, div, etc.) implemented as an independent Verilog module.
- Developed a **top-level ALU** (`alu_top.v`) integrating all modules with a **3-bit opcode decoder** for operation selection.
- Created a **comprehensive testbench** (`alu_tb.v`) using **nested loops** to test all 256 possible input combinations of A and B, and all logic operation modes. In this document, we have attached the screenshots for only one value of A and B each for easy understanding.
- Ensured that all modules are **combinational** for fast simulation and straightforward waveform analysis.
- Verified outputs using **functional waveforms** in Vivado 2022.2

3. Description of Each Module:

Addition Module:

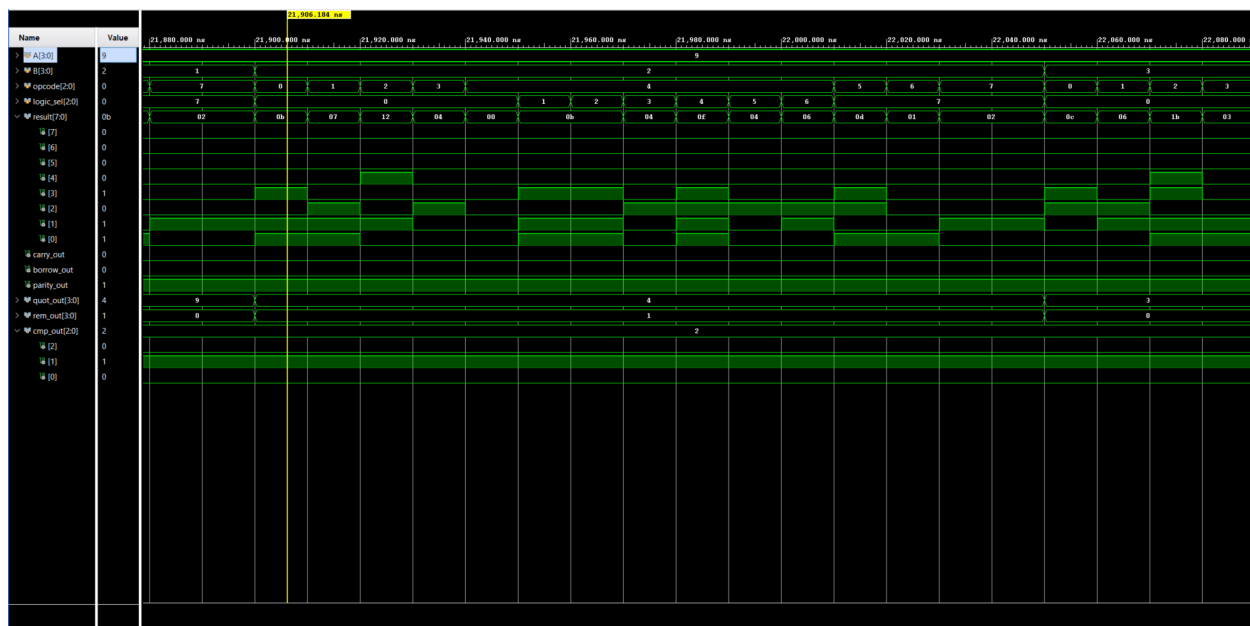
Function: Performs 4-bit addition with carry-out using a ripple-carry adder structure. Used in arithmetic units and accumulators.

Waveform Explanation:

In the simulation waveform, as inputs A and B vary, the output SUM increases accordingly. When the total exceeds binary 1111 (15 in decimal), the **carry_out** flag goes high, indicating an overflow beyond the 4-bit limit.

Use Case: Commonly used in ALUs and digital calculators.

Waveforms:



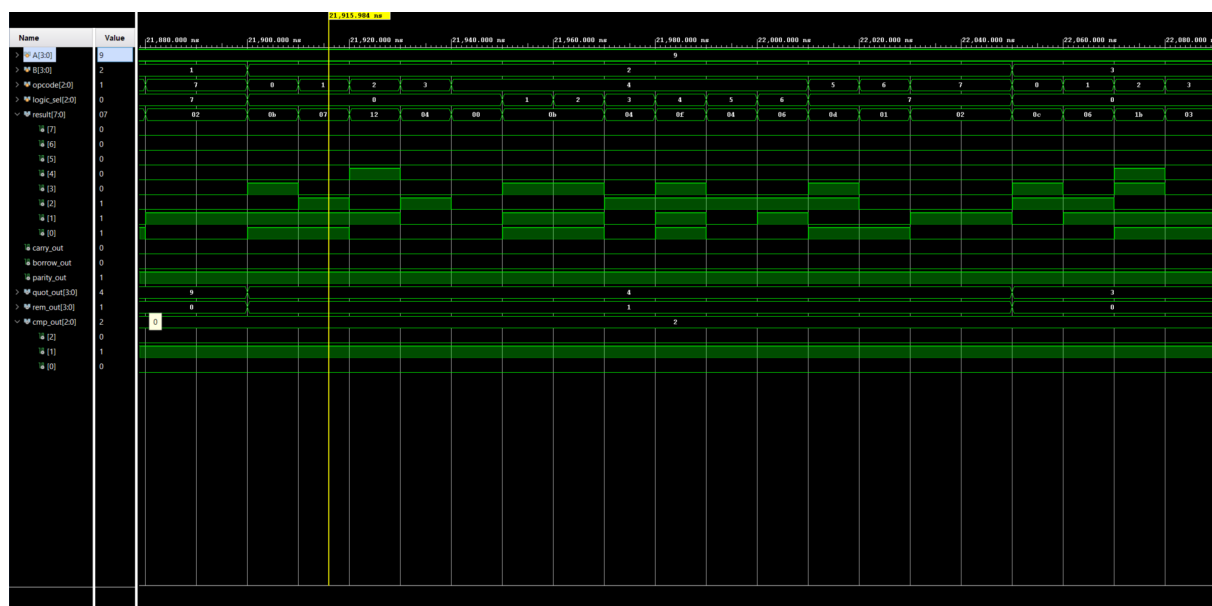
Here, A=1001 (9) and B=0010 (2) and the sum is output as '0b' (11) and carry=0

Subtraction Module:

Function: Performs 4-bit subtraction with borrow-out. Implemented using 2's complement addition logic.

Use Case: Used in arithmetic difference computation and control algorithms.

Waveforms:



Here, A=1001 (9) and B=0010 (2), and the difference is output as '0111' (7) and borrow=0

Waveform Explanation:

In the waveform, the **DIFF** signal shows the binary subtraction result.

When A becomes smaller than B, the **borrow_out** signal asserts (1), representing that a borrow was needed.

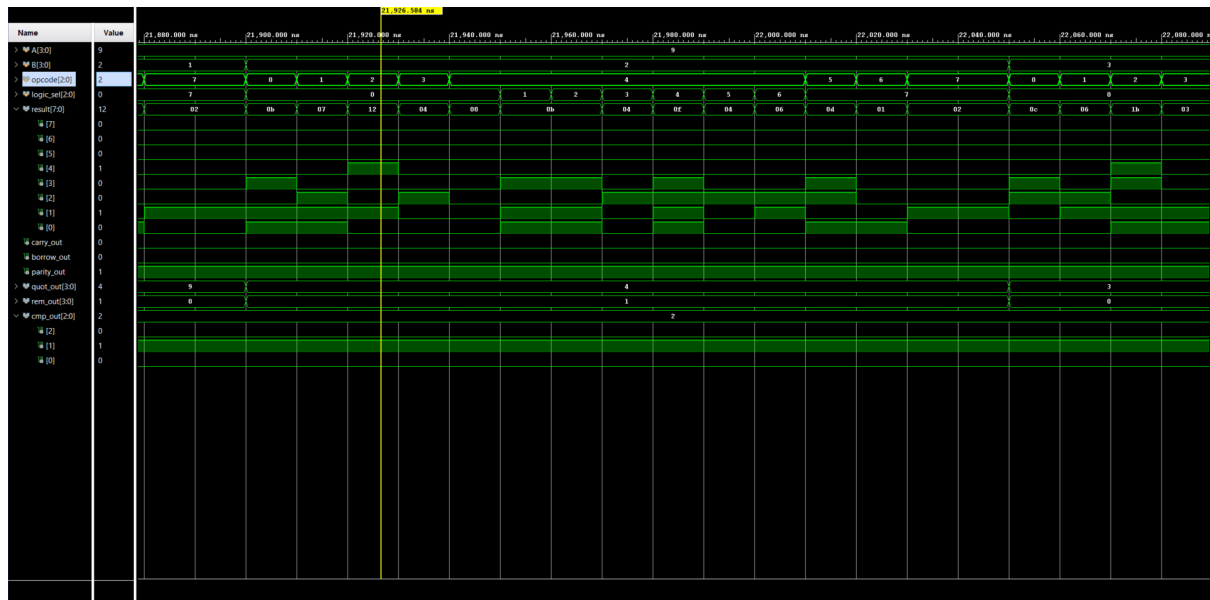
For instance, when A= and B=0101, DIFF=1101 (which is -3 in 2's complement) and borrow_out=1.

Multiplication Module:

Function: Performs 4×4 multiplication producing an 8-bit product output. Implemented via combinational partial products.

Use Case: Used in DSPs and embedded mathematical computations.

Waveforms:



Here, $A=1001$ (9) and $B=0010$ (2), and the Product is output as '00010010' (18 in hexadecimal is 12).

Waveform Explanation:

The `PROD` output shows an 8-bit result corresponding to the product of A and B .

Division Module:

The `div` module performs **integer division** between two 4-bit unsigned binary numbers A (dividend) and B (divisor).

It produces two 4-bit outputs:

- `quot_out` → Quotient (A divided by B)
- `rem_out` → Remainder (A modulo B)

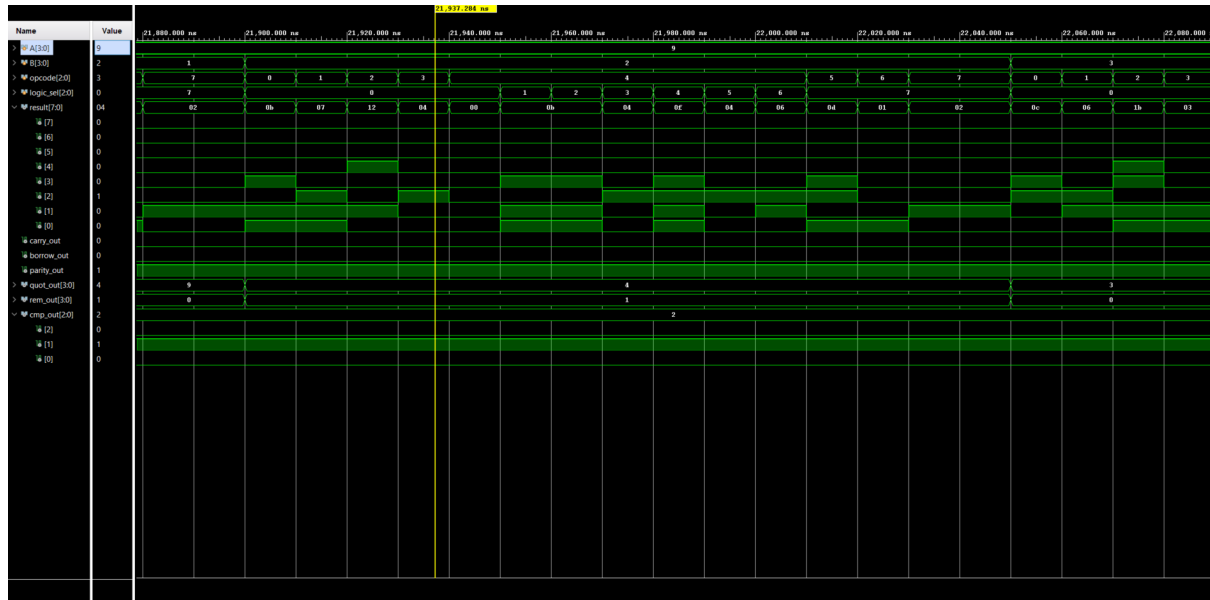
A check ($B \neq 0$) ensures that **division by zero** does not occur.

If $B = 0$, both outputs are set to 0000.

Use case: Used in **Arithmetic Logic Units (ALUs)** for division operations, found in **microprocessors, DSPs, and FPGA arithmetic blocks**.

Waveform Explanation:

- Two 4-bit inputs A and B are applied.
- When B is nonzero, the output shows integer division results.
- Both QUOT (quotient) and REM (remainder) outputs update **combinationally** whenever inputs change.



Here, A=1001 (9) and B=0010 (2), and the Division is output as '0010' (4) and quotient is 4, remainder is 1.

Logical Operations Module:

The `logic` module performs **seven fundamental bitwise logical operations** on two 4-bit binary inputs A and B.

It uses a **3-bit selection line** `logic_sel` to choose which logical operation to execute. Each output bit of `OUT` corresponds to the result of the selected logic operation applied bitwise between A and B.

Waveform explanation:

- A and B are applied as 4-bit binary values.
- The `logic_sel` signal is varied from 000 to 110.
- The `OUT` signal changes instantly to reflect the corresponding logical operation.

Use Case:

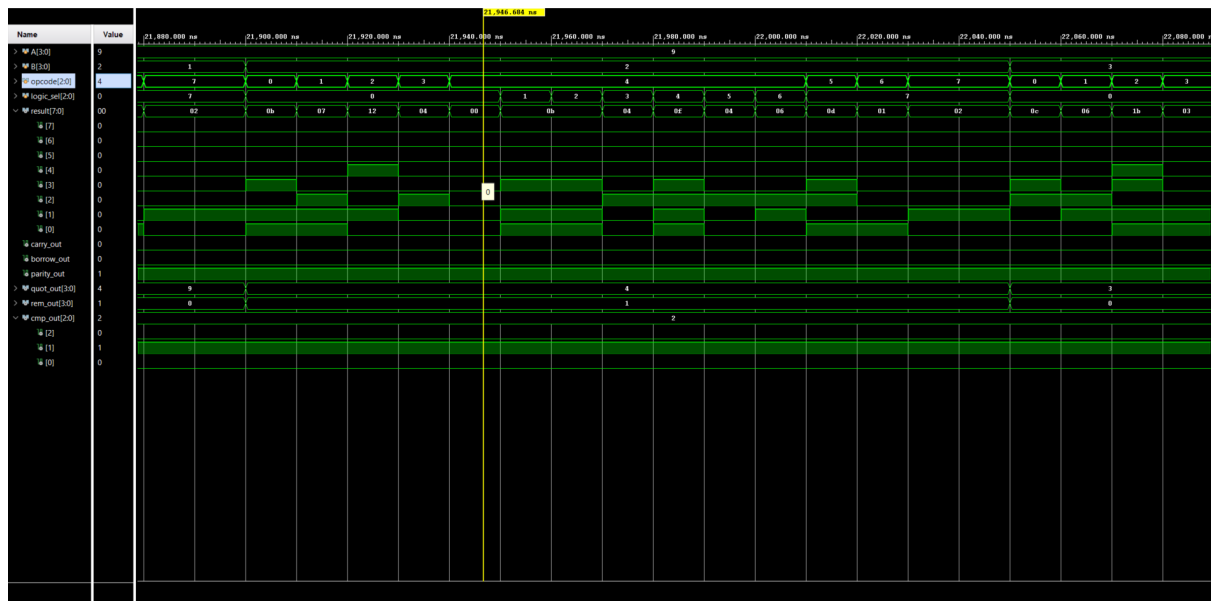
Used in **Arithmetic Logic Units (ALUs)** as the logical operation section, forming the **bitwise operation block** in CPUs and microcontrollers.

Supported Operations:

logic_sel	Operation	Description
000	AND	Output is 1 only if both A and B bits are 1
001	OR	Output is 1 if any corresponding bit of A or B is 1
010	XOR	Output is 1 if bits differ
011	XNOR	Output is 1 if bits are equal
100	NAND	Inverted AND
101	NOR	Inverted OR
110	NOT(A)	Bitwise inversion of A only

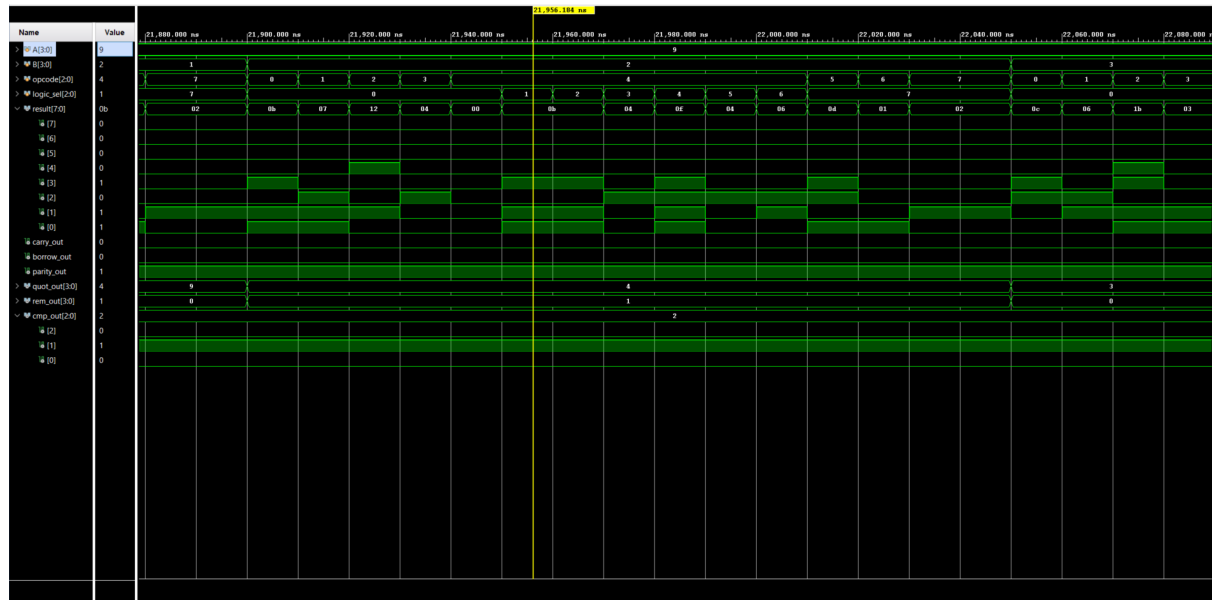
Waveforms:

AND Operation:



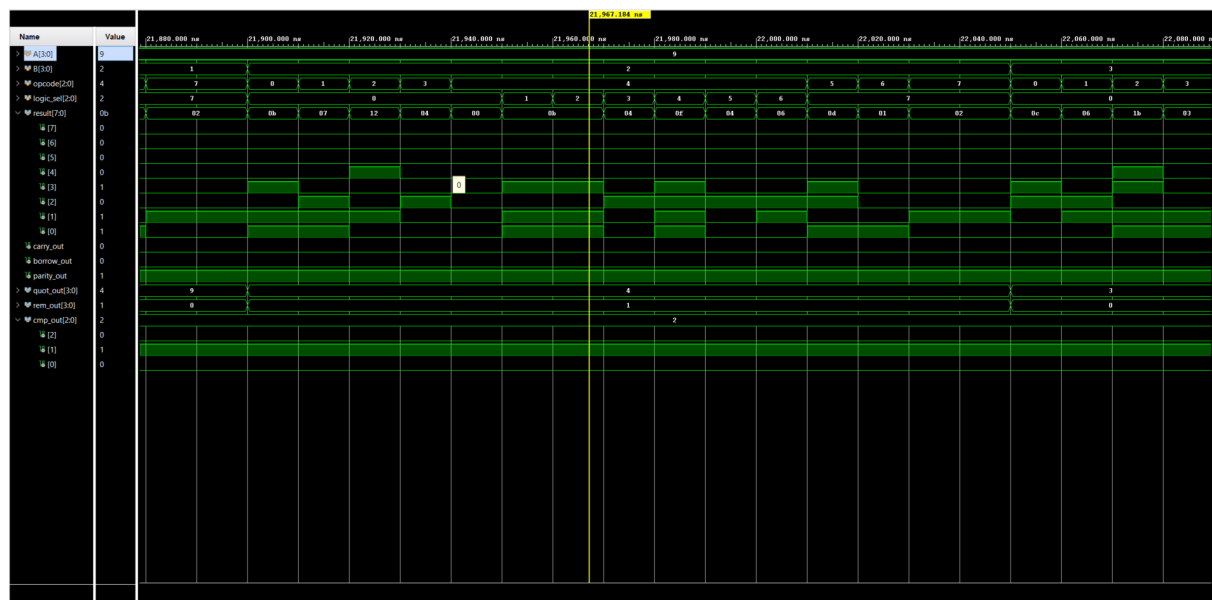
Here, A=1001 (9) and B=0010 (2), and the AND operation is output as '0000' (0)

OR Operation:



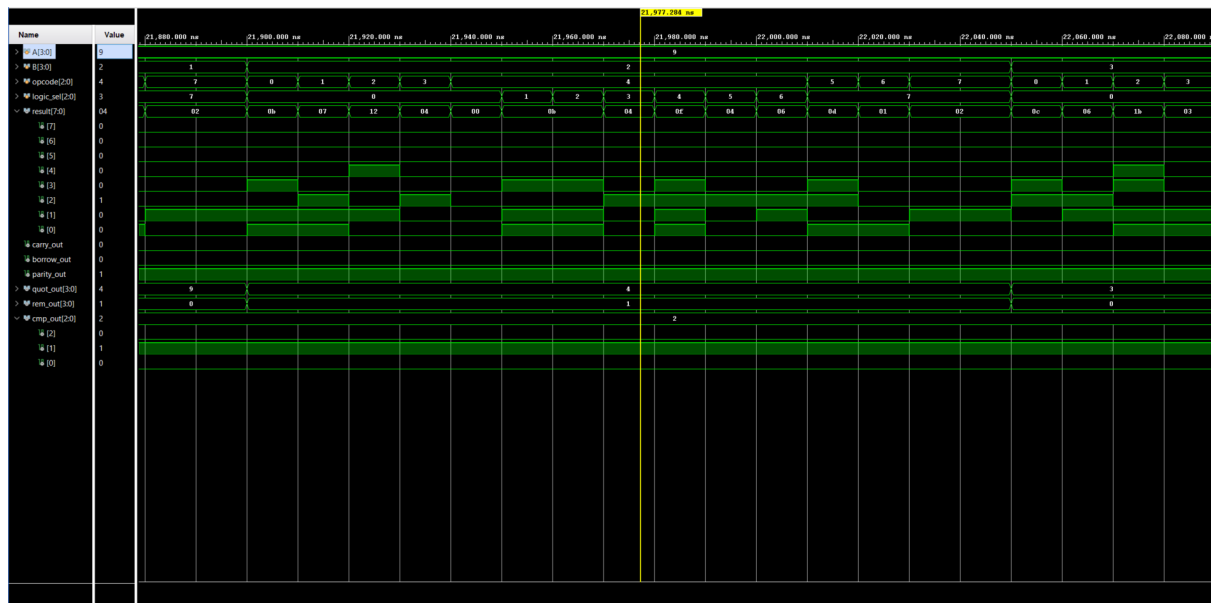
Here, A=1001 (9) and B=0010 (2), and the OR operation is output as '0b' (11).

XOR Operation:



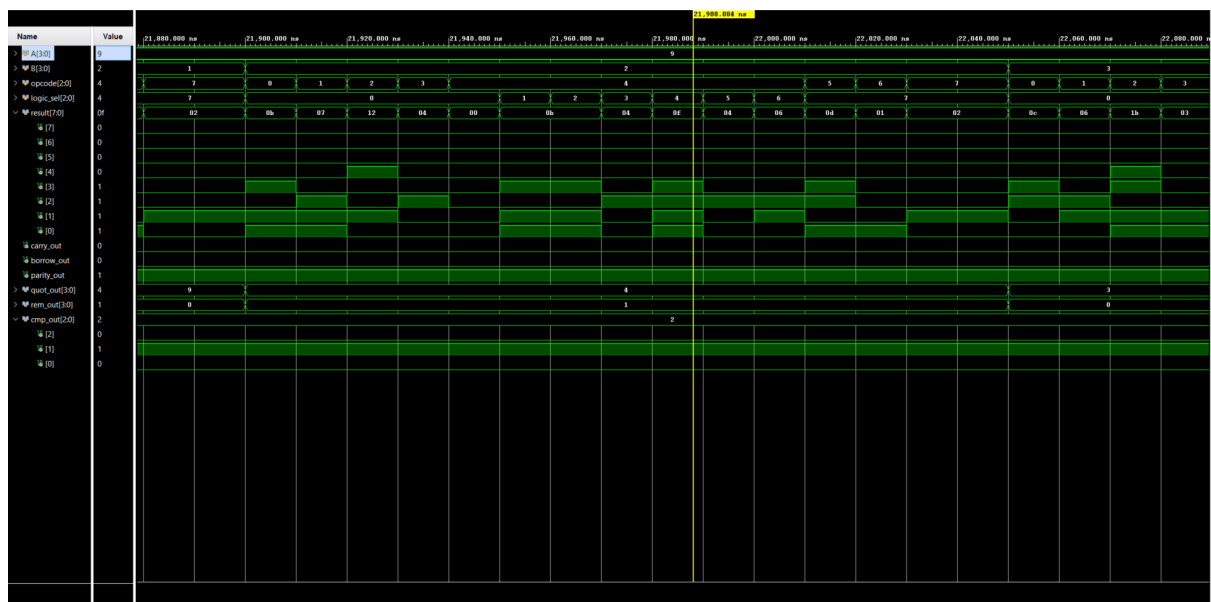
Here, A=1001 (9) and B=0010 (2), and the XOR operation is output as '0b' (12).

XNOR Operation:



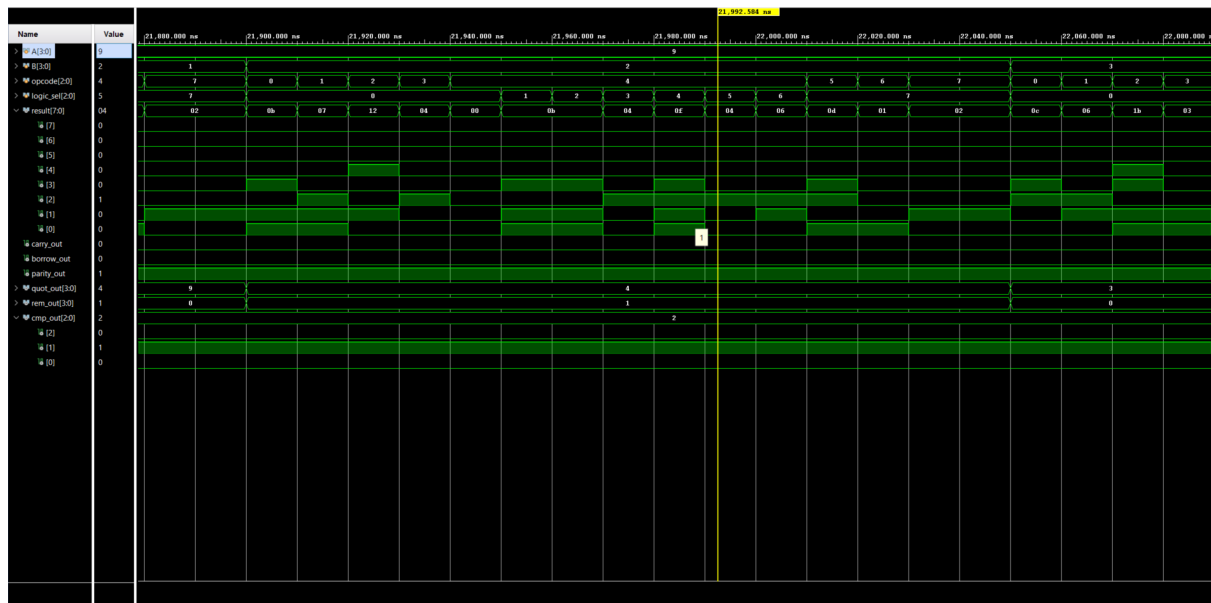
Here, A=1001 (9) and B=0010 (2), and the XNOR operation is output as '0100' (4).

NAND Operation:



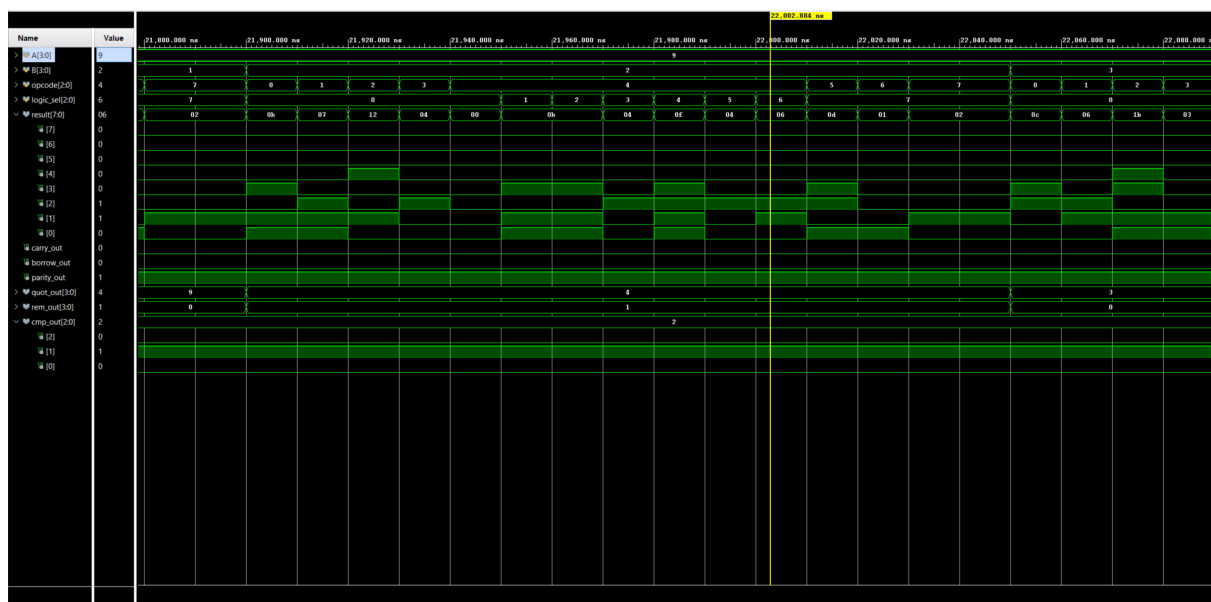
Here, A=1001 (9) and B=0010 (2), and the NAND operation is output as '0f' (15).

NOR Operation:



Here, A=1001 (9) and B=0010 (2), and the NOR operation is output as '0100' (4)

NOT Operation:



Here, A=1001 (9) and B=0010 (2), and the NOT operation is output as '0110' (6)

Gray Code Converter Module:

Function: Converts binary input to Gray code using XOR-based encoding logic. Reduces bit error sensitivity in transitions.

Waveform Explanation:

The G output toggles only one bit between consecutive input values of A.

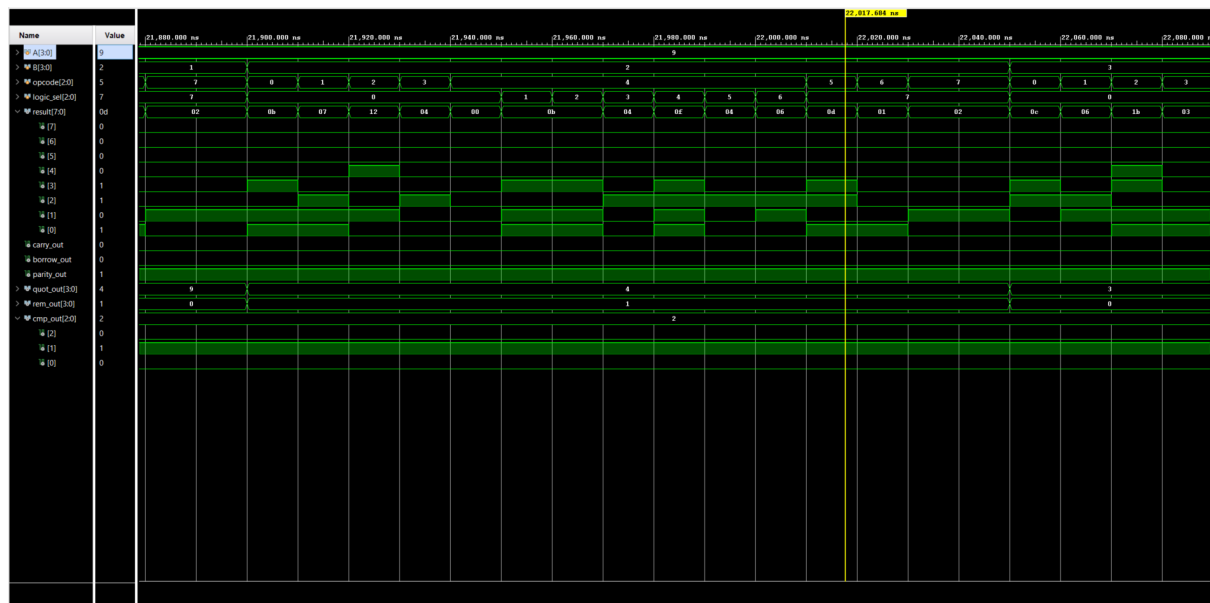
For instance:

- $A=0000 \rightarrow G=0000$
- $A=0001 \rightarrow G=0001$
- $A=0010 \rightarrow G=0011$
- $A=0011 \rightarrow G=0010$

This verifies correct Gray code behaviour — the waveform confirms single-bit transitions.

Use Case: Used in encoders, robotics, and communication systems.

Waveforms:



Here, $A=1001$ (9) and the corresponding Grey Code is output as '0d' (13)

Parity Generator Module:

Function: Generates even parity bit for 4-bit input using XOR reduction operator for error detection.

Waveform Explanation:

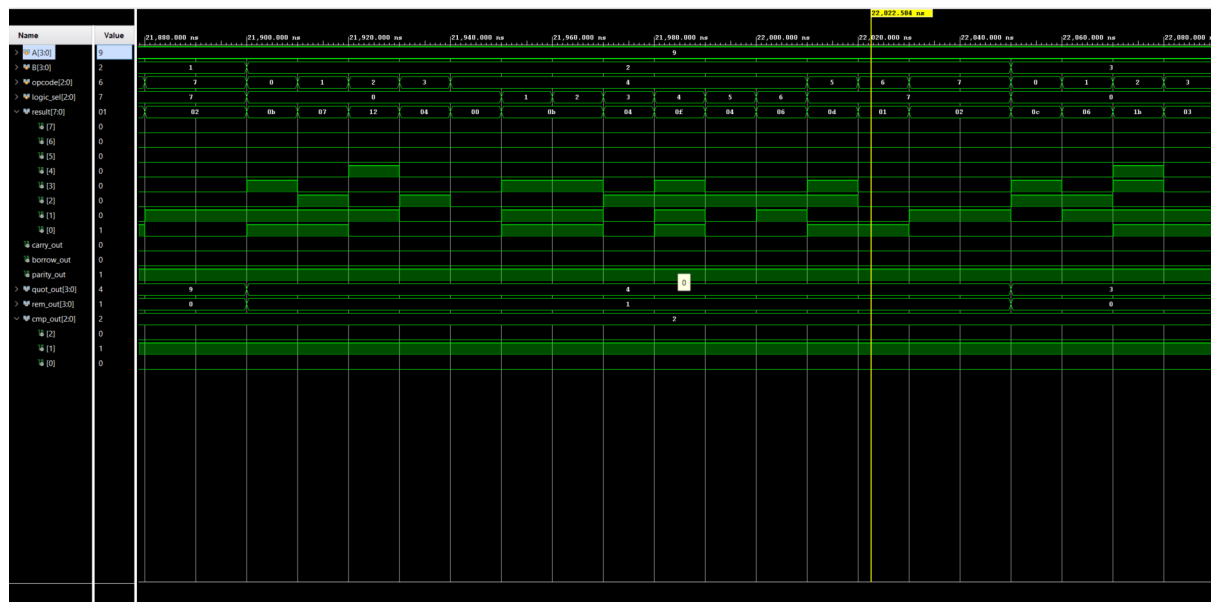
In the waveform, whenever the number of 1's in A is even, parity_out=1; if odd, parity_out=0.

For example:

- A=1011 (3 ones) → parity_out=0
 - A=1111 (4 ones) → parity_out=1
- This toggling pattern confirms correct parity generation.

Use Case: Used in digital communication systems for error detection.

Waveforms:



Here, A=1001 (9), and the parity generated is output as '1' since the number of set (1) bits in A is even.

Comparator Module:

Function: Compares two 4-bit inputs and outputs flags for equality, greater, and lesser conditions.

Waveform Explanation:

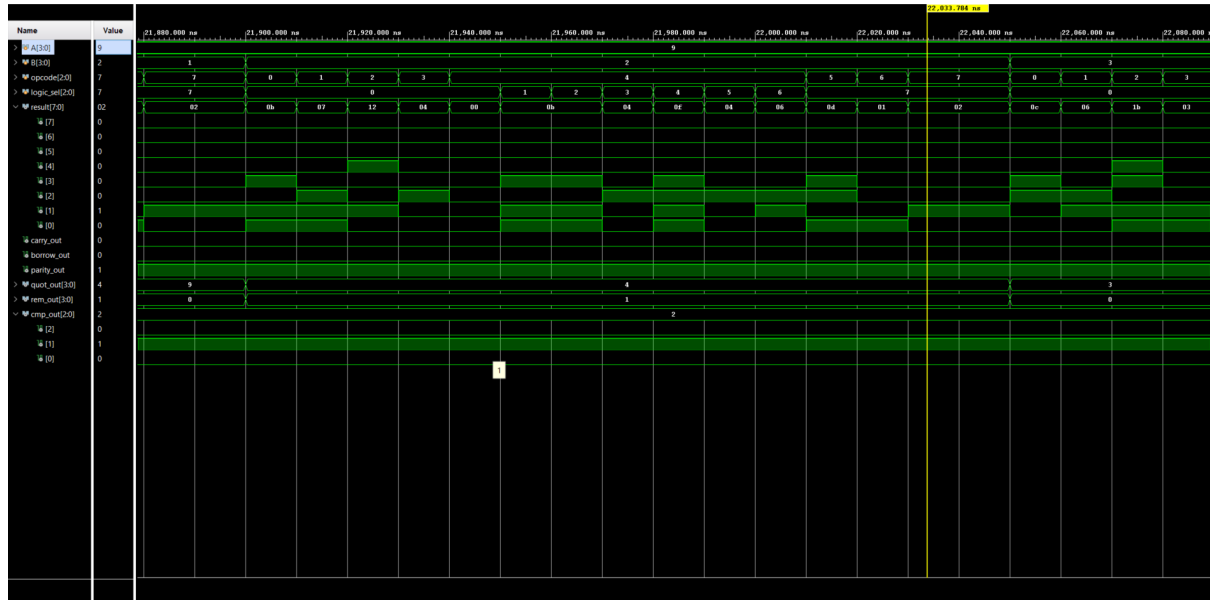
The waveform shows only one flag high at a time based on the comparison result:

- When A equals B , only $CMP[0] = 1$.
- When $A > B$, only $CMP[1] = 1$.

- When $A < B$, only $\text{CMP}[2] = 1$.
These transitions clearly verify the comparator's accuracy during simulation.

Use Case: Used in processors and decision-making control circuits.

Waveforms:



Here, $A=1001$ (9) and $B=0010$ (2). When compared, the output shows '010' (2), which is assigned to $A > B$ in the Verilog code.

Top Module:

Description:

The **top module (alu_top)** serves as the **central integration unit** that combines all eight functional submodules of the ALU into a single coherent system.

It takes two 4-bit inputs (A and B), a 3-bit **opcode** to select the desired operation, and a 3-bit **logic select line** for choosing among the bitwise operations.

Based on the opcode, it routes the input signals to the corresponding submodule and produces an 8-bit output ($result$) along with relevant status flags such as $carry_out$, $borrow_out$, $quot_out$, rem_out , $parity_out$, and cmp_out .

This modular integration approach makes the ALU **scalable**, **easy to debug**, and **structurally clear** for synthesis or future enhancements.

Function:

The alu_top module functions as a **multiplexer-based control unit** that:

1. Decodes the **3-bit opcode** to select one of the eight available operations:

- Addition
 - Subtraction
 - Multiplication
 - Division
 - Logic Operations
 - Gray Code Conversion
 - Parity Generation
 - Comparator
2. Instantiates each operation as a **separate submodule**, ensuring clean hierarchical design.
 3. Collects and forwards the output of the active module to the main **8-bit result bus**, with associated status or flag signals.
 4. Ensures seamless **operation switching** without conflicts between modules, enabling accurate simulation and waveform validation.

4. Learning Outcomes of the Mini ALU Design Project

1. **Comprehensive Understanding of ALU Architecture:**
Learned how multiple arithmetic and logical units (addition, subtraction, multiplication, division, logic, parity, Gray code, comparator, etc.) can be integrated into a single, modular ALU design using Verilog HDL.
2. **Modular Design and Hierarchical Integration:**
Understood the importance of **modular design methodology**, where each arithmetic or logical function was implemented as an independent Verilog module and then interconnected through a **top-level control structure** using opcode-based selection.
3. **Combinational Logic Design Principles:**
Strengthened understanding of **combinational circuits**, dataflow modelling (`assign` statements), and conditional operation selection (`case` and ternary operators) for implementing multi-functional digital blocks.
4. **Simulation and Functional Verification:**
Developed practical experience in verifying the correctness of digital designs using **testbenches and waveform analysis (VCD simulation)**.
Learned how to interpret transitions, check outputs, and validate multiple operations within a single simulation.
5. **Application-Oriented Thinking:**
Understood how each ALU sub-module corresponds to real-world applications — for example, parity in communication, Gray code in encoders, comparator in control units, and logic operations in control-flow execution.
6. **System-Level Design Thinking:**
Gained insight into how **individual functional blocks combine to form a complete processing unit**, mirroring how processors, DSPs, and controllers execute arithmetic and logical operations in parallel.

5. Conclusion:

The designed **4-bit ALU** demonstrates complete functionality, modular design, and comprehensive simulation coverage.
It performs eight distinct operations covering arithmetic, logical, and data encoding

categories, and the results validate its correctness through waveform analysis. The project reinforces core concepts in digital design, HDL implementation, and verification methodology

